

DOCUMENTACIÓN TÉCNICA MÓDULO 1: SYNTHCITY (SPRINT 1)

1. INTRODUCCIÓN Y OBJETIVO

La finalidad de este módulo es construir la base estructural completa y robusta del proyecto SynthCity. El sistema representa una ciudad modular en memoria garantizando la coherencia interna y la consistencia de todas las operaciones realizadas sobre el tablero.

2. DECISIONES DE DISEÑO ESTRUCTURAL

2.1. Gestión del Espacio y Tablero

- **Representación mediante Matriz:** El tablero se ha implementado como una matriz bidimensional de objetos Bloque.
- **Justificación Técnica:** Se ha elegido la matriz sobre otras estructuras porque el problema es espacial, las coordenadas son discretas y permite el acceso directo por fila y columna en tiempo constante.
- **Dimensiones de Ciudad:** Cada ciudad define sus propias dimensiones (filas y columnas) al ser creada, respetando un límite global del sistema para evitar crecimientos descontrolados.

2.2. Modelo de Dominio

- **Clase Posicion:** Se ha encapsulado la coordenada en una clase propia para aportar robustez. Es inmutable para asegurar que una posición no cambie una vez creada.
- **Clase Bloque (Abstracta):** Se define como abstracta para evitar la creación de bloques genéricos sin especialización.
- **Jerarquía de Bloques:** Se han implementado subclases específicas para cada tipo funcional: Residencial, Energia, Industrial, Servicios y Transporte.
- **Tipo Enumerado:** Se utiliza TipoBloque para evitar el uso de cadenas de texto libres y prevenir errores de consistencia.

3. INVARIANTES Y REGLAS DE NEGOCIO

El sistema garantiza las siguientes reglas de integridad de forma estricta:

1. **Unicidad de Celda:** No puede haber dos bloques ocupando la misma celda simultáneamente.
2. **Validación de Límites:** Cualquier operación de acceso o inserción debe verificar que las coordenadas estén dentro del grid (de 0 a filas-1 y de 0 a columnas-1).

3. **Integridad del Bloque:** No se permiten bloques nulos ni incompletos dentro del sistema.
4. **Estado Operativo:** Los bloques incluyen un estado activo/inactivo para soportar futuras simulaciones.

4. GESTIÓN DE EXCEPCIONES

Se han definido excepciones propias para comunicar errores del dominio de forma precisa, facilitando la depuración y la integración con otros módulos:

- **PosicionFueraDeLimitesException:** Se lanza cuando una coordenada no pertenece al tablero.
- **CeldaOcupadaException:** Se activa al intentar insertar un bloque en una posición ya ocupada.
- **CeldaVacíaException:** Ocurre al intentar eliminar o acceder a una celda que no contiene un bloque.
- **BloqueNuloException:** Garantiza que no entren referencias nulas en la estructura de la ciudad.

5. API DE CONSULTA Y UTILIDADES

Para desacoplar el almacenamiento de la lógica de procesamiento, el modelo ofrece:

- **Listados Dinámicos:** Métodos como `listarBloques()` y `listarBloquesActivos()` que devuelven colecciones (List) para que otros módulos procesen datos sin conocer la estructura de la matriz interna.
- **Métodos de Conteo:** Capacidad de obtener el total de bloques o la distribución por tipo, fundamental para futuras métricas y simulaciones.
- **Transparencia:** Representaciones textuales (`toString()`) diseñadas para ofrecer una visión rápida y funcional del estado de la ciudad y sus componentes.

6. PLAN DE PRUEBAS Y VALIDACIÓN ESTRUCTURAL

Para garantizar que el modelo es robusto y no se rompe bajo ningún escenario razonable, se han verificado los siguientes casos de prueba mínimos obligatorios:

6.1. Pruebas de Creación y Dimensiones

- **Creación de ciudad válida:** Se verificó la instancia correcta de ciudades con dimensiones dentro de los rangos permitidos.
- **Dimensiones inválidas:** Se comprobó que el sistema lanza una excepción (`IllegalArgumentExcepción`) al intentar crear ciudades con filas o columnas menores o iguales a cero.

6.2. Pruebas de Gestión de Bloques

- **Inserción en posición válida:** Se confirmó que los bloques se posicionan correctamente en las coordenadas indicadas por el objeto Posicion.
- **Inserción en posición ocupada:** Se validó que el sistema lanza una CeldaOcupadaException al intentar sobrescribir un bloque existente, protegiendo la integridad del tablero.
- **Inserción fuera de límites:** Se verificó el lanzamiento de PosicionFueraDeLimitesException al intentar añadir bloques en coordenadas negativas o superiores al tamaño del grid.
- **Eliminación de bloques:** Se probó la eliminación de bloques existentes y el control de errores (lanzamiento de CeldaVacíaException) al intentar eliminar en posiciones vacías.

6.3. Pruebas de Consulta y Estado

- **Conteo total y por tipo:** Se validó que los métodos contarBloques() y contarBloquesPorTipo() reflejan fielmente el estado del tablero tras múltiples operaciones.
- **Listado de bloques:** Se comprobó que listarBloques() devuelve una colección con todos los bloques existentes, facilitando el trabajo a futuros módulos sin exponer la matriz interna.
- **Filtro de activos:** Se verificó que listarBloquesActivos() filtra correctamente según el estado del bloque, preparando el sistema para el ciclo de simulación.

6.4. Verificación de Invariantes de Objetos de Valor

- **Igualdad de Posiciones:** Se comprobó mediante pruebas unitarias que el método equals de la clase Posicion identifica correctamente coordenadas idénticas, incluso en objetos con distinta referencia de memoria.
- **Consistencia de Hash:** Se verificó que objetos que son iguales según equals generan el mismo código hash, garantizando la compatibilidad con futuras colecciones.

7. CONCLUSIÓN DEL SPRINT

El sistema entregado cumple con el objetivo de proporcionar un modelo de ciudad robusto y seguro. Todas las validaciones estructurales están implementadas, asegurando que el Módulo 2 (Simulación) y el Módulo 3 (Métricas) puedan trabajar sobre una base que no se rompa bajo ningún escenario razonable.